

Reporte de Patrones de Diseño

Red Social de Personajes de Videojuegos

Brandon Juárez Zapata

8 de mayo de 2026

1. Introducción

Este reporte explica los patrones de diseño que se usaron en el proyecto *Personajes Videojuegos*. El proyecto tiene dos partes: la aplicación móvil hecha con Flutter (lo que el usuario ve) y el servidor hecho con Spring Boot (la parte que procesa la información). Para cada solución aplicada se explica: qué es y para qué sirve, en qué parte del código se usó, y qué ventaja nos da.

Las soluciones que se mencionan aquí se llaman “patrones de diseño”. Son recetas conocidas que los programadores usan para resolver problemas comunes de forma ordenada y reutilizable.

2. Resumen de las soluciones usadas

La siguiente tabla muestra de un vistazo todas las soluciones aplicadas, en qué parte del proyecto se usaron y para qué sirvieron:

Solución (Patrón)	¿Dónde se usó?	¿Para qué sirvió?
Singleton	App móvil y servidor	Garantizar que ciertos servicios existan una sola vez en toda la app
Service Locator	App móvil	Obtener servicios desde un punto central sin depender de clases específicas
Repository	App móvil y servidor	Separar la forma de guardar y leer datos del resto del código
Factory Method	App móvil y servidor	Crear objetos de forma controlada y centralizada
Builder	App móvil	Construir objetos complejos paso a paso de manera legible
Observer	App móvil	Notificar automáticamente cuando algo cambia en la app

State	App móvil	Controlar en qué estado se encuentra la app (cargando, listo, error, etc.)
Strategy	App móvil y servidor	Elegir cómo procesar algo según la situación
Command	App móvil	Encapsular acciones del usuario como dar “me gusta” o “no me gusta”
Template Method	App móvil	Reutilizar la misma estructura de pasos en operaciones similares
Specification / Query Object	App móvil	Hacer búsquedas con criterios específicos de forma reutilizable
DTO (objeto de transferencia)	Servidor	Enviar solo la información necesaria al cliente, sin datos internos

3. Patrones usados en la app móvil (Flutter)

3.1. Singleton — Una sola instancia

¿Dónde se usa?

- Archivo de autenticación: `AuthService.dart`
- Archivo de personajes: `PersonajeService.dart`
- Coordinador de servicios: `service_locator.dart`
- Estado general de la app: `app_state.dart`

¿Cómo funciona? Cuando la app arranca, crea una sola copia de ciertos servicios importantes (como el de autenticación o el de datos de personajes). Si alguien pide ese servicio más tarde, siempre recibe la misma copia, nunca una nueva.

¿Por qué es útil? Evita que se creen varias copias del mismo servicio al mismo tiempo, lo cual podría causar errores o consumir memoria innecesaria.

3.2. Service Locator — El directorio de servicios

¿Dónde se usa? Archivo: `service_locator.dart`

¿Cómo funciona? Es como una agenda o directorio: registra todos los servicios disponibles y cuando una pantalla los necesita, simplemente los pide por nombre sin saber cómo están contruidos por dentro.

¿Por qué es útil? Las pantallas y componentes visuales no dependen directamente de los servicios concretos. Esto facilita cambiar un servicio real por uno de prueba sin tocar el código de las pantallas.

3.3. Repository — Separar cómo se guardan los datos

¿Dónde se usa?

- Contrato (interfaz): `PersonajeRepository.dart`
- Implementación real (conectada al servidor): `PersonajeService.dart`
- Implementación de prueba (datos simulados): `PersonajeServiceMock.dart`

¿Cómo funciona? La app define un “contrato” que dice qué operaciones se pueden hacer con los personajes (obtener, crear, editar, eliminar). Hay una versión que realmente habla con el servidor y otra que usa datos de prueba. La interfaz visual siempre habla con el contrato, no con la versión específica.

¿Por qué es útil? Hace que sea muy fácil probar la app sin necesidad de conectarse a internet, y permite cambiar la fuente de datos sin modificar las pantallas.

3.4. Factory Method — Fábrica de objetos

¿Dónde se usa?

- Método `Personaje.fromJson(...)`
- Método `PersonajeBuilder.fromJson(...)`
- Servicios que convierten respuestas del servidor en objetos de la app

¿Cómo funciona? Cuando el servidor devuelve información en formato JSON (texto estructurado), existe un método especial que lee ese texto y crea el objeto correcto de la app automáticamente.

¿Por qué es útil? Toda la lógica de “leer datos del servidor y convertirlos” está en un solo lugar. Si el formato cambia, solo hay que modificar ese método.

3.5. Builder — Constructor paso a paso

¿Dónde se usa? Archivo: `personaje_builder.dart`

¿Cómo funciona? En lugar de crear un personaje en una sola línea complicada, se usa un constructor que va agregando la información poco a poco: primero el nombre, luego el videojuego, luego la imagen, etc.

¿Por qué es útil? Hace que el código sea más fácil de leer y evita errores al crear objetos con muchos campos.

3.6. Observer + State — Notificaciones y estados de la app

¿Dónde se usa? Archivo: `app_state.dart`

¿Cómo funciona? La app puede estar en distintos estados: iniciando, cargando, con sesión activa, sin sesión, con error, etc. Cuando el estado cambia, todas las pantallas que estaban “escuchando” se actualizan solas.

¿Por qué es útil? El comportamiento de la app es predecible y ordenado. No hay que actualizar cada pantalla manualmente: se actualizan solas cuando algo importante cambia.

3.7. Strategy — Elegir cómo procesar según la situación

¿Dónde se usa? Método `_parsePersonajeResponse(...)` en `PersonajeService`.

¿Cómo funciona? A veces el servidor devuelve los datos de una forma, a veces de otra. Este patrón revisa qué formato llegó y elige automáticamente cómo procesarlo:

- Si la respuesta viene como lista paginada (con la clave `content`), la procesa de una manera.
- Si viene como lista directa, la procesa de otra.

¿Por qué es útil? La app funciona correctamente aunque el servidor cambie ligeramente cómo devuelve los datos.

3.8. Command — Encapsular acciones del usuario

¿Dónde se usa? Acciones `likePersonaje(...)` y `dislikePersonaje(...)`.

¿Cómo funciona? Cada vez que el usuario presiona “me gusta” o “no me gusta”, esa acción queda encapsulada en una función específica con un solo objetivo.

¿Por qué es útil? Es más fácil rastrear qué hizo el usuario y manejar lo que sucede después (actualizar contador, mostrar mensaje, etc.).

3.9. Template Method — Estructura común para operaciones similares

¿Dónde se usa? Operaciones de crear, actualizar y eliminar personajes.

¿Cómo funciona? Todas las operaciones siguen los mismos pasos en el mismo orden:

- Preparar la petición.
- Enviarla al servidor.
- Verificar si la respuesta fue exitosa.
- Convertir la respuesta en objetos de la app.
- Manejar cualquier error que ocurra.

¿Por qué es útil? Todos los servicios de red funcionan de la misma manera, lo que hace el código más fácil de entender y mantener.

3.10. Specification / Query Object — Búsquedas específicas

¿Dónde se usa?

- `getPersonajeById(int id)` — buscar un personaje por su número de identificación.
- `searchByVideojuego(String videojuego)` — buscar personajes por nombre de videojuego.

¿Por qué es útil? Cada tipo de búsqueda está claramente definida y puede reutilizarse en varios lugares de la app.

3.11. Value Object — Objetos que representan datos

¿Dónde se usa? Modelos Personaje y Comentario.

¿Cómo funciona? Los modelos de la app representan exactamente lo que significan: un personaje tiene nombre, imagen, videojuego, etc. Se tratan como representaciones fijas de esa información.

¿Por qué es útil? Reduce el riesgo de cambiar datos por accidente y hace que el código sea más claro sobre qué información se está manejando.

4. Patrones usados en el backend (Spring Boot)

4.1. Repository con Spring Data JPA

¿Dónde se usa?

- `PersonajeRepository` (para personajes)
- Repositorios de comentarios, usuarios y reacciones

¿Cómo funciona? Spring Boot ofrece herramientas que generan automáticamente las operaciones de base de datos más comunes (buscar, guardar, eliminar, listar). Solo hay que decirle qué tipo de dato manejar.

¿Por qué es útil? Se escribe mucho menos código repetitivo. La lógica de guardar datos queda separada de la lógica de negocio de los controladores.

4.2. DTO — Objeto de transferencia de datos

¿Dónde se usa?

- `PersonajeResponseDTO.java` — datos de un personaje para enviar al cliente.
- `ComentarioResponseDTO.java` — datos de un comentario.
- `ReactionResponseDTO.java` — datos de una reacción (me gusta / no me gusta).

¿Cómo funciona? Cuando el servidor responde a la app, no envía la entidad completa de la base de datos. En cambio, crea un objeto limpio con solo la información que la app necesita, incluyendo campos calculados como el número de “me gusta” y si el usuario actual ya reaccionó.

¿Por qué es útil? Las respuestas son más seguras, más ligeras y no exponen información interna de la base de datos.

4.3. Factory Method en el backend

¿Dónde se usa? Método `convertToDTO(Personaje personaje, Long userId)` en `CharacterController`.

¿Cómo funciona? Este método toma un personaje de la base de datos y lo convierte en un DTO completo, calculando contadores y estados de reacción en ese momento.

¿Por qué es útil? Toda la lógica de construcción de respuestas está en un solo lugar, lo que facilita mantener la consistencia entre todos los endpoints de la API.

4.4. Strategy en el backend — Cálculo de reacciones

¿Dónde se usa? Filtros aplicados en `CharacterController` para contar me gusta, no me gusta, y determinar si el usuario actual ya reaccionó.

¿Cómo funciona? Se aplican diferentes reglas de cálculo según el tipo de reacción y si el usuario autenticado ya reaccionó previamente.

¿Por qué es útil? La lógica de conteo está clara y separada, y puede ajustarse sin afectar el resto del controlador.

5. Conclusión

A lo largo del desarrollo del proyecto *Red Social de Personajes de Videojuegos*, usar estas soluciones de programación demostró ser una buena decisión que mejoró la calidad del código y facilitó el trabajo del equipo. Contar con patrones como Singleton, Factory Method y Builder permitió crear objetos de forma ordenada y sin errores, mientras que Repository y Service Locator lograron que las pantallas, los servicios y el manejo de datos pudieran modificarse por separado sin que un cambio en una parte rompiera las demás. Esto que al principio puede parecer un detalle menor, en la práctica significó ahorrarse muchos dolores de cabeza durante el desarrollo.

Los patrones relacionados con el comportamiento de la app, como Observer, State, Strategy y Command, aportaron claridad a situaciones que de otra forma habrían sido difíciles de controlar, como saber en qué estado se encuentra la app en cada momento o qué debe pasar exactamente cuando el usuario presiona “me gusta” o “no me gusta”. De hecho, esas funciones fueron las que más problemas presentaron al principio, con errores en los contadores y en el formato de los datos. Gracias a tener el código bien organizado fue posible encontrar y corregir esos errores sin tener que reescribir todo desde cero.

En el servidor, enviar al cliente solo la información necesaria mediante los DTOs, en lugar de mandar todos los datos internos de la base de datos, resolvió varios problemas de formato y dejó las respuestas mucho más limpias y seguras. En general, todas estas decisiones dejaron al proyecto en una posición mucho más cómoda para el futuro: agregar nuevas funciones, corregir errores o hacer pruebas es ahora una tarea más sencilla y menos riesgosa que al inicio, lo cual es exactamente el objetivo que se busca al aplicar buenas prácticas de programación desde el principio.

En conjunto, el proyecto quedó mejor preparado para crecer y para ser probado en el futuro.